

How to deploy trustmebro

For architecture and API details, see `CLIENT_TECHNICAL_DOCUMENTATION.md` and the repository `README.md`.

This guide is for anyone hosting the **trustmebro** Next.js app on a remote Linux server (for example a campus machine) **without** administrator (`sudo`) access. It also explains how to fill in `.env.local` with real API keys and points to short tutorials for Git and npm when you need to deploy or redeploy.

Getting API keys for `.env.local`

Environment variables are **NEVER** committed to Git. On your laptop you create `.env.local` in the project root; on the server you do the same after cloning. The full list of variables, with comments, lives in `.env.local.example`. Copy that file as a starting point:

```
cp .env.local.example .env.local
```

Then edit `.env.local` with `nano .env.local` (or any text editor) and replace the placeholder values below.

PMXT (prediction market data)

1. Open PMXT and sign in (or create an account).
2. Go to the PMXT dashboard.
3. Create or copy an API key. Live keys typically start with `pmxt_live_`.
4. In `.env.local`, set:

```
PMXT_API_KEY=pmxt_live_your_actual_key_here
```

Upstash Redis (caching and rate limits)

1. Open Upstash Console and sign in.
2. Create a **Redis** database (the free tier is fine for many school projects).
3. Open the database and find **REST URL** and **REST TOKEN** (sometimes labeled as connection details for the REST API).
4. In `.env.local`, set:

```
UPSTASH_REDIS_REST_URL=https://...upstash.io  
UPSTASH_REDIS_REST_TOKEN=your_token_here
```

Anthropic (chat features)

1. Open Anthropic Console and sign in.
2. Add credits to your account. The chatbot API is **not** free for use.
3. Go to **API keys** and create a new key.
4. In `.env.local`, set:

```
ANTHROPIC_API_KEY=sk-ant-your_actual_key_here
```

Optional variables

See `.env.local.example` for optional settings such as `ANTHROPIC_MODEL`, chat rate limits, `PMXT_ARCHIVE_URL`, and `CRON_SECRET` (if you use scheduled warmup routes in production).

Security tips: Do not paste keys into chat, email, or public repos. If a key leaks, revoke it in the provider's dashboard and create a new one.

Learn the commands (Git and npm)

Use these official guides when you are new to the terminal or need a refresher:

Topic	Where to learn
Installing Node.js and npm	Node.js download page and npm — Downloading and installing Node.js and npm
What npm <code>install</code> does	npm install reference
Cloning and updating a repo	Git — Cloning a repository and Git — Pulling changes
GitHub Personal Access Token (private repos)	Managing your personal access tokens (GitHub Docs)

The **deploy / redeploy** flow on the server always boils down to: `git pull` → `npm install` (if dependencies changed) → `npm run build` → restart the process manager (PM2), as shown below.

Server deployment guide

This section explains how to host **trustmebro** on a remote Linux server using a non-root account.

1. Prerequisites

- An SSH login to the server.
- Node.js and npm installed on the server (see the table above if you need help).
- A **GitHub Personal Access Token** if the repository is private (see GitHub link above). When `git clone` asks for a password, use the token instead of your GitHub password.

2. Clone the repository

Connect to your server via SSH and enter your password:

```
ssh [user]@[ip_address]
```

and run:

```
git clone https://github.com/jasmina-dev/trustmebro.git
cd trustmebro
```

If the repo is private, Git may prompt for credentials; use your GitHub username and a **personal access token** as the password.

3. Environment configuration

The `.env.local` file is not tracked by Git. Create it on the server so APIs and caching work:

```
cp .env.local.example .env.local
nano .env.local
```

Paste your real values (see Getting API keys for `.env.local` above). At minimum you normally need `PMXT_API_KEY`, `UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`, and `ANTHROPIC_API_KEY`.

Press `Ctrl+O`, then `Enter`, then `Ctrl+X` to save and exit `nano`.

4. Install and build

Install dependencies and create the production build:

```
npm install
npm run build
```

5. Process management (PM2)

Because students often lack `sudo`, install PM2 locally (project dependency) and run it with `npx`.

Start the application:

```
# Replace 8002 with your assigned or open port  
PORT=8002 npx pm2 start npm --name "trustmebro" -- start
```

Essential PM2 commands:

- **Check status:** `npx pm2 list`
- **View logs:** `npx pm2 logs trustmebro`
- **Restart app:** `npx pm2 restart trustmebro`
- **Stop app:** `npx pm2 stop trustmebro`
- **Delete process:** `npx pm2 delete trustmebro`

6. Scheduled cache warmup (cron)

The route `GET /api/warmup` pre-fills Redis with the same cache keys the dashboard charts need (markets aggregates, paginated market slices, divergence categories, resolution bias, and related data). When Redis is warm, real users trigger fewer **misses** and your server stays closer to the PMXT rate limits.

On **Vercel**, `vercel.json` can schedule this route automatically. On a **Linux server you manage yourself**, nothing triggers warmup unless **you** schedule it (for example with `cron`).

Requirements:

1. **CRON_SECRET in .env.local** — use any long random string (generate one locally; it is not issued by a vendor). The Next.js app must see this variable when it runs (`next start` under PM2 loads `.env.local` from the project root).
2. **HTTP header on every cron request** — production requires:

Authorization: Bearer <same value as CRON_SECRET>

Manual check (replace port and secret):

```
curl -fsS -H "Authorization: Bearer YOUR_CRON_SECRET_HERE" "http://127.0.0.1:8002/api/warmup"
```

You should get JSON such as `"warmed": true` and timing fields. A JSON error about `CRON_SECRET` means the server process does not have the variable set, or the Bearer token does not match.

Example crontab (every five minutes, aligned with the Vercel schedule in `vercel.json`). Run `crontab -e` and add one line; replace the port with your `PORT` and avoid putting the secret directly in the file if you prefer a small env script:

```
*/5 * * * * . /home/youruser/trustmebro/.cron-env.sh && curl -fsS -m 420 -H "Authorization:
```

Create `.cron-env.sh` next to the project (or under your home directory) with:

```
export CRON_SECRET='your-long-random-secret'
```

Then restrict permissions: `chmod 600 .cron-env.sh`.

Flags:

- `-m 420` gives `curl` a seven-minute ceiling so a stuck run does not pile up forever (warmup can legitimately take minutes).

Use `http://127.0.0.1:PORT` if the cron runs on the same machine as PM2; use your public `https://` URL only if you intend to hit the site through your reverse proxy.

7. Updating the app (redeploy)

To pull new changes from GitHub and refresh the live site:

```
git pull
npm install      # Only needed if package.json changed
npm run build    # Rebuild the project
npx pm2 restart trustmebro
```

8. Troubleshooting

- **Port already in use:** If port 8002 is taken, run `lsof -i :8002` (if available) to find the process, stop it with `kill -9 <PID>`, or pick another port in the `PORT=...` command.
- **Processes die when you disconnect:** If the server kills background jobs when you log out, run your session inside `screen` or `tmux` so PM2 keeps running, or ask your administrator about a proper login service.

If the site does not load in the browser at `http://<server-ip>:8002`, confirm with your campus IT that inbound traffic is allowed on that port.